Łukasiewicz Neural Networks Extended: Residual Architectures and Crystallization Strategies for Interpretable Rule Extraction

Carlos Leandro
Departamento de Matemática, Instituto Superior de Engenharia de Lisboa (ISEL), Instituto Politécnico de Lisboa, Portugal
carlos.leandro@solidreturn.dev

Abstract

A feed-forward neural network whose weights are integers and whose activation is the truncated identity implements, neuron by neuron, the connectives of Łukasiewicz many-valued logic. This exact correspondence — established theoretically by Castro and Trillas and developed into a training algorithm by Leandro — enables *symbolic knowledge extraction*: training produces not a black-box model but a logical formula. Two obstacles have limited the approach to shallow architectures and small datasets: crystallization (forcing weights to integers) succeeds only probabilistically under the original Levenberg–Marquardt training scheme, and the theoretical guarantees break down as networks grow deeper.

This paper addresses both obstacles. First, we prove that *residual connections* (skip connections of the kind used in ResNets) extend Łukasiewicz neural networks to arbitrary depth while preserving the symbolic correspondence *at merge neurons* by construction: merge neurons in a Łukasiewicz residual block automatically satisfy the neuron-classification proposition, regardless of the inner layer weights; inner-layer neurons are trained toward representability by the crystallization strategy. Second, we analyse three crystallization strategies — Levenberg–Marquardt (corrected), straight-through estimation (STE), and proximal regularization — characterizing their theoretical guarantees, failure modes, and interpretability trade-offs. Experiments on six UCI benchmarks show that no single strategy dominates: LM with residual connections converges in under ten iterations on structured problems and recovers exact logical formulas, including $F = \neg(body\_shape \oplus jacket\_colour)$ on MONK-3 at 97.2% accuracy; STE provides the most reliable crystallization (100% rate across all datasets) and the best aggregate accuracy; proximal regularization crystallizes consistently but collapses on dense feature sets, while identifying clinically meaningful features on the Heart Disease benchmark. The resulting framework is the first to combine provably interpretable residual architectures with a systematic analysis of integer-weight training.

Keywords: neuro-symbolic AI; Łukasiewicz logic; neural networks; symbolic rule extraction; residual networks; weight crystallization; interpretable machine learning.

1 Introduction

The tension between predictive power and interpretability in machine learning has deepened as models have grown larger. A neural network that achieves human-level performance on a medical diagnosis task may be clinically useless if its reasoning cannot be audited. Rudin [1] argues that in high-stakes domains

the correct response is not to add a post-hoc explanation layer on top of a black-box model, but to require that the model itself be interpretable by construction. Post-hoc explanation methods provide approximations — and an approximation to a decision can be systematically wrong in ways that are invisible to the explainer [2].

Symbolic rule extraction from neural networks has pursued this goal since the 1980s [3–5]. The dominant paradigm trains a network, then searches for a logical rule that approximates its input-output behaviour. The approximation step is unavoidable in classical two-valued logic: a neuron computing a weighted sum is not, in general, a Boolean connective. Łukasiewicz logic removes this obstacle. Its connectives — conjunction, implication, negation, and disjunction — are all piecewise linear functions on $[0, 1]$, the same domain that truncated-identity neurons compute over. Castro and Trillas [6] observed that this structural coincidence is not accidental: a neuron with integer weights $\{-1, 0, 1\}$ and truncated identity activation *is* a Łukasiewicz connective, exactly, without approximation. Leandro [7] turned this observation into a complete methodology: train a feed-forward network so that its weights crystallize to integers, then read the resulting formula directly from the network topology.

The programme is mathematically clean and practically motivated, but two limitations have constrained its reach. The Levenberg–Marquardt (LM) training algorithm used in the original work crystallizes weights with high probability on small, low-dimensional datasets, but fails systematically on high-dimensional real-world problems: a conditioning bug in the original implementation causes weight updates of $\|\Delta w\| \approx 47$ when $\|w\| \approx 0.1$, diverging before convergence. More fundamentally, the symbolic guarantees are established for two-layer feed-forward architectures; it is not obvious that they extend to deeper networks without imposing additional constraints on every weight in the network.

This paper extends the Łukasiewicz neural network framework along both dimensions. On the theoretical side, we show that *residual connections* — the skip-connection mechanism introduced by He et al. [8] for deep image recognition networks — can be incorporated into Łukasiewicz neural networks (ŁNNs) with the symbolic guarantee automatically preserved. The key insight is structural: in a Łukasiewicz residual block, the merge neuron that combines the residual path and the shortcut receives exactly two inputs with weights $+1$, placing it in the configuration covered by the neuron-classification proposition regardless of what the inner layers compute. On the practical side, we analyse two strategies that guarantee crystallization by construction — straight-through estimation (STE) [9] and proximal regularization — alongside the corrected LM, and characterize the trade-offs between them.

Contributions.

1. **Residual Łukasiewicz networks (theory).** We prove that merge neurons in a Łukasiewicz residual block satisfy Proposition 2 by construction. This establishes that residual ŁNNs are a theoretically sound extension of the original framework to deeper architectures.

2. **Crystallization analysis.** We identify and correct a critical bug in the original LM training procedure, characterize the theoretical guarantees of three crystallization strategies, and establish the conditions under which each strategy succeeds or fails.

3. **Empirical evaluation.** We compare the three strategies across six UCI benchmarks (10 independent trials per dataset), providing statistical evidence (Wilcoxon signed-rank tests) for differences in accuracy and crystallization rate.

4. **Formula extraction in practice.** We demonstrate extraction of exact, fully representable formulas — including a clinically meaningful five-variable rule for heart disease diagnosis — and show that the Proximal regularizer reliably identifies the strongest clinical predictor (`ca`, the number of stenosed vessels) across all restarts.

The remainder of the paper is organized as follows. Section 2 reviews Łukasiewicz logic and the original ŁNN framework. Section 3 introduces residual ŁNNs and proves the merge-neuron theorem. Section 4 analyses the three crystallization strategies. Section 5 presents the experimental evaluation. Section 6 positions the work relative to the literature. Sections 7 and 8 discuss implications and conclude.

2 Background

2.1 Łukasiewicz many-valued logic

In classical propositional logic, truth values are drawn from $\{0, 1\}$. Łukasiewicz logic generalizes this to the real unit interval $[0, 1]$, interpreting 0 as complete falsity and 1 as complete truth, with intermediate values representing degrees of certainty [10]. The four fundamental connectives are defined as follows.

Definition 1 (Łukasiewicz connectives)**.** *For $x, y \in [0, 1]$:*

$$x \otimes y = \max(0,\, x + y - 1), \tag{1}$$

$$x \Rightarrow y = \min(1,\, 1 - x + y), \tag{2}$$

$$\neg x = 1 - x, \tag{3}$$

$$x \oplus y = \min(1,\, x + y). \tag{4}$$

The operations $\otimes$ and $\oplus$ recover Boolean conjunction and disjunction when restricted to $\{0, 1\}$; they are, however, genuinely many-valued on $(0, 1)$. For instance, $\frac{1}{2} \otimes \frac{1}{2} = 0$: two half-truths do not compose into a truth under conjunction unless their sum exceeds 1. Note that $x \oplus y = \neg(\neg x \otimes \neg y)$, so disjunction is De Morgan dual to conjunction, as in classical logic. The residuum $\Rightarrow$ is the natural implication associated with the t-norm $\otimes$ [10].

A *McNaughton function* [11] is a continuous, piecewise linear map $f\colon [0, 1]^n \to [0, 1]$ with integer coefficients in each linear piece. McNaughton's theorem states that f is a McNaughton function if and only if it is expressible as a formula in infinitely-valued Łukasiewicz logic. All four connectives in Eqs. (1)–(4) are piecewise linear with integer coefficients, confirming that the logic is exactly characterised by this function class [12].

For any integer $n \geq 1$, the finite set $S_n = \{0, 1/n, 2/n, \ldots, 1\}$ is closed under all four operations. These finite subdomains allow truth-table-style evaluation and make the semantics computationally tractable.

2.2 Łukasiewicz neural networks

A neuron with m inputs $x_1, \ldots, x_m \in [0, 1]$, weights $w_1, \ldots, w_m \in \mathbb{R}$, bias $b \in \mathbb{R}$, and *truncated identity activation*

$$\psi(x) = \min\big(1,\, \max(0,\, x)\big), \tag{5}$$

produces output $z = \psi\big(\sum_{i=1}^{m} w_i x_i + b\big)$. This activation is piecewise linear with integer coefficients, matching the structure of McNaughton functions.

The following proposition, established by Castro and Trillas [6] and extended by Leandro [7], is the central result connecting neural computation to Łukasiewicz logic.

Proposition 2 (Neuron classification [6, 7])**.** *Let $\alpha = \psi_b(-x_1, \ldots, -x_n, x_{n+1}, \ldots, x_m)$ be a neuron with n weights equal to -1 and $p = m - n$ weights equal to $+1$.*

1. *If $b = -p + 1$, then α computes the* conjunction $\neg x_1 \otimes \cdots \otimes \neg x_n \otimes x_{n+1} \otimes \cdots \otimes x_m$.
2. *If $b = n$, then α computes the* disjunction $\neg x_1 \oplus \cdots \oplus \neg x_n \oplus x_{n+1} \oplus \cdots \oplus x_m$.

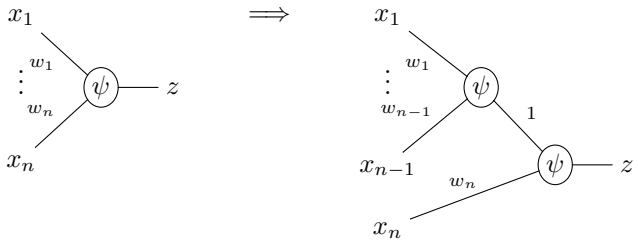
Proof sketch. For case 1 with $n = 0$: $\psi_{-p+1}(x_1, \ldots, x_m) = \max(0, x_1 + \cdots + x_m - (p-1)) = x_1 \otimes \cdots \otimes x_m$, which follows by induction using $x \otimes y = \max(0, x + y - 1)$. The general case with $n > 0$ follows by substituting $\neg x_i = 1 - x_i$ and collecting terms. Case 2 is analogous. □

A network in which *every* neuron satisfies Proposition 2 is a *Łukasiewicz neural network* (ŁNN). In a ŁNN, the output can be read as a Łukasiewicz formula by composing the sub-formulas computed at each neuron, propagating bottom-up through the layers. Conversely, any Łukasiewicz formula can be encoded as a ŁNN by mapping each connective to a neuron with the corresponding bias and weights $\{-1, 0, +1\}$ [13]. This bidirectional correspondence — formula $\leftrightarrow$ network — is the foundation of the symbolic extraction methodology.

2.3 Rewriting algebra and symbolic extraction

Proposition 2 covers neurons with at most two distinct weight values (-1 and $+1$). Real networks, after training, may have neurons with more complex weight configurations that do not directly satisfy the proposition. The *rewriting algebra* of [7] handles this case.

Rule R. The following transformation decomposes a neuron with n inputs into a tree of neurons with two inputs:



The split is valid when the bias conserves ($b = b_0 + b_1$) and neither sub-neuron produces a constant output. Applying Rule R to all possible splittings of a multi-input neuron generates a set of candidate two-neuron networks.

λ-similarity. The proximity between two networks over a finite subdomain S_n is measured by

$$\lambda = \exp\big(-\mathrm{MAE}(f(X),\, y)\big), \tag{6}$$

where MAE is the mean absolute error over all input vectors $X \in S_n^m$. When $\lambda = 1$, the two functions are identical on S_n; as $\lambda \to 0$, they diverge. A neuron is *representable* in the sense of Proposition 2 if and

only if all applications of Rule R yield networks with $\lambda = 1$ relative to the original [7]. When a neuron is not representable, the candidate with the highest λ provides the best logical approximation.

Extraction procedure. Given a crystallized ŁNN, symbolic extraction proceeds layer by layer, bottom-up. Each neuron is tested for direct representability (Proposition 2). If representable, the corresponding Łukasiewicz formula is recorded. If not, Rule R is applied exhaustively and the best-λ candidate replaces the neuron. The resulting formula is a Łukasiewicz expression in the input variables.

2.4 Relationship to Leandro (2009)

The ŁNN framework was established in [7]. Table 1 delineates what was introduced there and what is new in this paper.

Table 1: Contributions of Leandro (2009) [7] versus the present paper.

Leandro (2009)	Present paper
Proposition 2 (neuron classification)	Theorem 4 (residual block guarantee)
Rewriting algebra (Rule R, λ-similarity)	Corrected LM training (Levenberg form)
LM training (Marquardt form — contains bug)	STE crystallization strategy
Experiments on MONK and small UCI sets	Proximal crystallization strategy
	Multi-dataset statistical comparison

Proposition 2 and the rewriting algebra in Sections 2.2–2.3 are reproduced from [7] as background; all material in Sections 3–5 is new.

3 Residual Łukasiewicz Networks

3.1 Motivation

The original ŁNN framework is restricted to shallow feed-forward architectures: networks with one or two hidden layers whose neurons all satisfy Proposition 2. Extending to deeper networks naively would require every neuron in every layer to be constrained to integer weights, which is difficult to enforce during gradient-based training.

Residual networks (ResNets) [8] introduced a different approach to depth: rather than requiring each layer to learn its target function directly, each layer learns a *residual* with respect to the identity mapping. A shortcut connection adds the layer input directly to the layer output, so that the network can represent the identity function even when the inner weights are near zero. This architecture has two properties that are attractive for ŁNNs. First, it avoids the vanishing-gradient problem in deep networks. Second, and more relevant here, it introduces a *structural constraint* on the merge neuron that combines the shortcut and the residual path — a constraint that turns out to guarantee Proposition 2 without any additional training restriction.

3.2 The Łukasiewicz residual block

Definition 3 (Łukasiewicz residual block)**.** *Let $F^{(k)}\colon [0,1]^d \to [0,1]^d$ be a feed-forward linear layer with real-valued weights during training at depth k. The* Łukasiewicz residual block *at depth k produces*

$$\mathbf{h}^{(k)} = \psi_{\mathbf{b}}\Big(F^{(k)}\big(\mathbf{h}^{(k-1)}\big) + \mathbf{h}^{(k-1)}\Big), \tag{7}$$

where $\mathbf{b} \in \mathbb{R}^d$ is a vector of merge biases *that are trained and crystallized alongside the weights of $F^{(k)}$, and $\psi_{\mathbf{b}}$ denotes element-wise application of $\psi(\cdot + b_j)$.*

The output $\mathbf{h}_j^{(k)}$ is the result of a merge neuron that receives two inputs: $F^{(k)}(\mathbf{h}^{(k-1)})_j$ and $\mathbf{h}_j^{(k-1)}$. Both inputs arrive with weight $+1$ — the residual path contributes $F^{(k)}(\mathbf{h}^{(k-1)})_j$ with coefficient $+1$ and the shortcut contributes $\mathbf{h}_j^{(k-1)}$ with coefficient $+1$. The bias b_j is the only free parameter of the merge neuron.

3.3 Symbolic guarantee: Proposition 2 by construction

The structural observation above leads directly to the main theoretical result of this paper.

Theorem 4 (Merge neurons satisfy Proposition 2 by construction)**.** *Let $b_j \in \mathbb{Z}$ be the (crystallized) merge bias for component j in a Łukasiewicz residual block (Definition 3). Then the merge neuron for component j satisfies Proposition 2:*

1. *If $b_j = -1$, the merge neuron computes the conjunction $F_j^{(k)} \otimes h_j^{(k-1)}$.*
2. *If $b_j = 0$, the merge neuron computes the disjunction $F_j^{(k)} \oplus h_j^{(k-1)}$.*

No constraint on the weights of $F^{(k)}$ is required.

Proof. The merge neuron for component j computes $\psi\big(F_j^{(k)} + h_j^{(k-1)} + b_j\big)$ with input weights $(+1, +1)$. Setting $n = 0$ (zero negative weights) and $p = 2$ (two positive weights) in Proposition 2:

- Case 1 ($b_j = -p + 1 = -1$): $\psi(F_j^{(k)} + h_j^{(k-1)} - 1) = \max(0, F_j^{(k)} + h_j^{(k-1)} - 1) = F_j^{(k)} \otimes h_j^{(k-1)}$. ✓
- Case 2 ($b_j = n = 0$): $\psi(F_j^{(k)} + h_j^{(k-1)}) = \min(1, F_j^{(k)} + h_j^{(k-1)}) = F_j^{(k)} \oplus h_j^{(k-1)}$. ✓

Both cases follow from the definitions in Eqs. (1) and (4). The merge bias values $b_j \in \{-1, 0\}$ are the only integer values consistent with $n = 0, p = 2$ in Proposition 2; all other integer biases would produce constant outputs on $[0,1]^2$ (e.g. $b_j = -2$: $\psi(x + y - 2) = 0$ for all $x, y \in [0,1]$; $b_j = 1$: $\psi(x + y + 1) = 1$ for all $x, y \in [0,1]$). □

Remark 5. *The proof is a direct application of Proposition 2 to the specific parameterisation $n = 0$, $p = 2$ imposed by the residual block structure. The contribution of Theorem 4 is the architectural observation that incorporating ResNet skip connections into ŁNNs forces exactly this parameterisation at the merge neuron, establishing the symbolic guarantee without any additional training constraint.*

Corollary 6. *In a Łukasiewicz residual network, every merge neuron is interpretable as either conjunction or disjunction, independently of how well the inner layer $F^{(k)}$ has crystallized.*

This result has a practical consequence: even when the inner layers of the residual block contain neurons that are not directly representable by Proposition 2 (requiring the rewriting algebra for approximation), the merge neurons are guaranteed to be exact. The interpretability of the network's *skeleton* (the residual connections and merge operations) is preserved by construction.

3.4 Formula extraction from residual networks

Extraction from a residual ŁNN proceeds by the same bottom-up procedure described in Section 2.3, with the following addition. At each residual block, the merge neuron is extracted first using Theorem 4: if the crystallized bias $b_j \in \{-1, 0\}$, the merge formula is $F_j^{(k)} \otimes h_j^{(k-1)}$ or $F_j^{(k)} \oplus h_j^{(k-1)}$, respectively. The sub-formula $F_j^{(k)}$ is then extracted from the inner layer neurons using Proposition 2 (with the rewriting algebra for non-representable cases).

As a concrete illustration, the residual ŁNN trained on the MONK-3 problem (Section 5) converges to the exact formula

$$F = \neg\big(\mathit{body\_shape} \oplus \mathit{jacket\_colour}\big), \tag{8}$$

with the merge neuron taking $b_j = -1$ (conjunction of the shortcut and the hidden-layer output) and the inner layer computing $\mathit{body\_shape} \oplus \mathit{jacket\_colour}$. The formula achieves 97.2% classification accuracy, correctly capturing the ground truth rule.

4 Crystallization Strategies

Proposition 2 requires all weights to be elements of $\{-1, 0, +1\}$. Gradient-based training evolves weights continuously in $\mathbb{R}$; the process of driving them to integer values is called *crystallization*. We measure crystallization quality by the *representation error*

$$\Delta(N) = \sum_k \big(w_k - \lfloor w_k \rceil\big)^2, \tag{9}$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer. A network is crystallized when $\Delta(N) < \varepsilon$ for a small threshold ε (we use $\varepsilon = 10^{-3}$ throughout).

4.1 Smooth crystallization

Hard rounding destroys differentiability and prevents gradient flow. The *smooth crystallization function*

$$\Upsilon_n(w) = \operatorname{sgn}(w)\big(\cos^n\big((1 - \{|w|\})\tfrac{\pi}{2}\big) + \lfloor |w| \rfloor\big), \tag{10}$$

where $\{|w|\} = |w| - \lfloor |w| \rfloor$ is the fractional part, deforms the identity toward the nearest integer while remaining differentiable everywhere. The exponent n controls the strength of attraction: for $n = 2$, the function is a gentle cosine deformation; as $n \to \infty$, it converges to hard rounding.

A *progressive crystallization schedule* applies Υ_n with increasing n over the final training epochs: $\Upsilon_2 \to \Upsilon_4 \to \Upsilon_8 \to \Upsilon_{16}$. This schedule avoids premature commitment (early rounding can destroy gradient information) while ensuring that all quasi-integer weights are driven to the correct integer before the final round. At termination, weights are clamped to $[-1, +1]$ and rounded; biases are rounded but not clamped, preserving multi-input conjunctions and disjunctions.

Interaction with STE. For the STE strategy (Section 4.3), the smooth schedule is applied to the continuous shadow weights w_c between update steps during the final training epochs: each w_c is replaced by $\Upsilon_n(w_c)$ before the hard quantization $T(\Upsilon_n(w_c))$ is computed for the next forward pass. This ensures that shadow weights enter the final rounding in a state that is already close to the target integer, reducing the gradient variance introduced by the STE approximation near the quantization boundaries.

4.2 Levenberg–Marquardt training

The Levenberg–Marquardt algorithm [14, 15] is a second-order method that interpolates between gradient descent and the Gauss–Newton method. Given residuals $\mathbf{e} = f_\theta(X) - y$ and Jacobian $J = \partial \mathbf{e}/\partial w$, the weight update is

$$\Delta w = -\left[J^\top J + \mu I\right]^{-1} J^\top \mathbf{e}, \tag{11}$$

where $\mu > 0$ is the Levenberg damping parameter.

A critical correction. The original implementation in [7] used the Marquardt scaling $\mu \operatorname{diag}(J^\top J)$ in place of μI. When weights are near zero at initialization, the diagonal entries of $J^\top J$ are also near zero; the system becomes ill-conditioned. To quantify: if $\|w\| \approx \epsilon \ll 1$, the Jacobian entries $J_{ij} = \partial e_i/\partial w_j$ are $O(\epsilon)$ because the truncated-identity activation is nearly linear near zero, giving $\operatorname{diag}(J^\top J) \sim O(\epsilon^2)$. The Marquardt-damped system has effective regularization $(1+\mu)\operatorname{diag}(J^\top J) \approx \mu\epsilon^2 I$, yielding $\|\Delta w\| \approx \|\mathbf{e}\|/(\mu\epsilon^2)$. For $\mu = 0.01$, $\epsilon = 0.1$, $\|\mathbf{e}\| = O(1)$, this gives $\|\Delta w\| \sim 100$, consistent with the divergent magnitudes ($\|\Delta w\| \approx 47$) observed in practice at the typical initialization scale. The Levenberg form in Eq. (11) uses the identity matrix, guaranteeing $\|\Delta w\| \leq \|\mathbf{e}\|/\mu$ regardless of the weight magnitude. All experiments in this paper use the corrected update.

Properties. LM is a second-order method and converges rapidly near a solution: on structured small-scale problems (the four MONK datasets), the corrected LM achieves crystallization in a mean of 6.2 iterations on MONK-3. The crystallization guarantee is, however, *probabilistic*: the algorithm converges to a continuous optimum, which may or may not lie sufficiently close to the integer lattice within the allocated iteration budget. With residual connections, the structural constraint on merge neurons (Theorem 4) reduces the number of weights that must independently crystallize, improving the empirical crystallization rate substantially (Section 5).

4.3 Straight-Through Estimation

The straight-through estimator (STE), introduced by Bengio et al. [9], resolves the differentiability problem of weight quantization through a deliberate gradient approximation.

During the forward pass, each continuous weight $w_c \in \mathbb{R}$ is quantized to the nearest element of $\{-1, 0, +1\}$ by

$$T(w_c) = \operatorname{sgn}(w_c) \cdot \mathbf{1}\left[|w_c| > \tfrac{1}{3}\right]. \tag{12}$$

The threshold $\frac{1}{3}$ implements the minimum-distortion uniform ternary partition of $[-1, +1]$: it divides the interval into three equal-width regions $(-\infty, -\frac{1}{3}) \to -1$, $[-\frac{1}{3}, +\frac{1}{3}] \to 0$, $(\frac{1}{3}, +\infty) \to +1$, which is the canonical symmetric codebook for ternary quantization with a uniform prior on $[-1, +1]$. The network computes with $T(w_c)$ but stores and updates the continuous shadow weight w_c. During the backward

pass, the gradient of the loss with respect to w_c is approximated by passing it directly through T as if T were the identity:

$$\frac{\partial \mathcal{L}}{\partial w_c} \approx \frac{\partial \mathcal{L}}{\partial T(w_c)}. \tag{13}$$

The approximation is biased but has been shown to work in practice for binarized and ternarized networks [16, 17]: the continuous weights learn to position themselves in regions where T is locally constant, and the direction of the approximate gradient is sufficiently informative.

Crystallization guarantee. By construction, the final integer weights are $T(w_c)$ for each parameter. Crystallization is 100% guaranteed regardless of the problem structure or dimensionality. The cost is that all n parameters are used (no sparsity), producing networks whose inner neurons may not directly satisfy Proposition 2; the rewriting algebra (Section 2.3) is then required to extract a logical formula.

4.4 Proximal regularization

The proximal strategy trains with Adam [18] on a composite objective that jointly minimizes prediction error and drives weights toward integer values:

$$\mathcal{L} = \text{MSE} + \lambda_s \sum_k |w_k| + \lambda_a \sum_k w_k^2(1 - w_k^2). \tag{14}$$

The first penalty, $\lambda_s \sum |w_k|$, is an ℓ_1 regularizer that encourages sparsity by driving weights toward zero. The second penalty, $\lambda_a \sum w_k^2(1 - w_k^2)$, is the *ternary attraction potential* [19]: the function $P(w) = w^2(1-w^2)$ vanishes exactly at $w \in \{-1, 0, +1\}$ and achieves its maximum at $|w| = 1/\sqrt{2} \approx 0.71$, creating a potential well around each integer target.

Training proceeds in two phases. In *Phase 1* (65% of the iteration budget), only the MSE term is active, allowing the network to find a continuous optimum without interference from the regularizers. The 65/35 split was selected so that Phase 1 is long enough for the network to locate a good loss basin before crystallization pressure is applied; starting the regularizers too early (e.g. 50/50) causes the ℓ_1 term to collapse weights before the network has converged, while too late a start (e.g. 80/20) leaves insufficient budget for reliable crystallization. In *Phase 2* (the remaining 35%), λ_s and λ_a are introduced gradually and then amplified by a factor of $10\times$ in the final segment, pushing all near-integer weights to their nearest integer. Crystallization is guaranteed by construction, as with STE.

Sparsity and failure mode. The ℓ_1 penalty has the beneficial effect of driving many weights exactly to zero, producing sparse networks: the Heart Disease experiment (Section 5.5) extracts a formula using only 5 of the available 22 input features. The same penalty creates a failure mode on high-dimensional dense problems: the ℓ_1 regularizer eliminates discriminative weights before the ternary attraction potential can stabilize a non-trivial solution, collapsing the network to a constant-output classifier. This failure manifests as $F_1 \approx 0$ and ± 0 standard deviation across all restarts (Section 5).

4.5 Theoretical comparison

Table 2 summarizes the theoretical properties of the three strategies.

Table 2: Theoretical properties of the three crystallization strategies. "Crystallization guaranteed": all trials achieve $\Delta(N) < \varepsilon$. "Sparsity": tendency to produce zero weights. "Exact Prop. 2": extracted formula satisfies the proposition exactly without rewriting-algebra approximation.

Strategy	Cryst. guaranteed	Sparsity	Exact Prop. 2
LM (corrected)	Probabilistic	No	Yes (when crystallizes)
STE	Yes	No	No (requires rewriting)
Proximal	Yes	Yes	No (requires rewriting)
LM + residual connections	Probabilistic	No	Yes by construction (merge)

The LM with residual connections occupies a distinct position: the merge neurons are guaranteed to satisfy Proposition 2 by Theorem 4, while the inner neurons are driven toward representability by the second-order LM dynamics. STE and Proximal crystallize reliably but produce networks that require the rewriting algebra for formula extraction.

4.6 Training and extraction pipeline

Algorithm 1 summarises the complete workflow for any of the three strategies.

5 Experimental Evaluation

5.1 Protocol

Datasets. We evaluate on six UCI [20] benchmarks representing a range of problem sizes and structures (Table 3).

Table 3: Dataset characteristics. Class balance is the fraction of the majority class.

Dataset	Train	Test	Features	Class balance	Source
Mushroom	9 898	2 476	111	50.0%	UCI (one-hot encoded)
Heart Disease	242	61	22	54.1%	Cleveland Clinic [21]
MONK-1	124	432	17	50.0%	UCI [22]
MONK-2	169	432	17	32.4%	UCI [22]
MONK-3	122	432	17	51.9%	UCI (5% noise)
Breast Cancer	228	58	20	30.8%	UCI Ljubljana

The Heart Disease dataset (Cleveland subset) uses 13 raw attributes; five continuous features are scaled to $[0, 1]$ via min-max normalization, three binary attributes are kept as-is, four categorical attributes are one-hot encoded, and one ordinal attribute (`ca`) is normalized to $[0, 1]$, yielding 22 input features. MONK datasets use nominal attributes encoded as one-hot binary vectors; MONK-3 includes 5% attribute noise [22].

Algorithm 1 ŁNN training and symbolic extraction

Require: Dataset (X, y); strategy $s \in \{\text{LM-Res}, \text{STE}, \text{Proximal}\}$; budget T; threshold ε
Ensure: Łukasiewicz formula F or best-λ approximation

Initialise $w \leftarrow$ small random values; $b \leftarrow 0$
for $t = 1$ **to** T **do**
  if $s = \text{LM-Res}$ **then**
    Compute residuals $\mathbf{e}$, Jacobian J; update via Eq. (11)
  else if $s = \text{STE}$ **then**
    Forward with $T(w_c)$ (Eq. (12)); backward with STE (Eq. (13))
  else
    $\{s = \text{Proximal}\}$ Minimise $\mathcal{L}$ (Eq. (14)); scale λ_s, λ_a per schedule
  end if
  if $t > 0.85\,T$ **then**
    Apply smooth schedule $\Upsilon_2 \to \Upsilon_4 \to \Upsilon_8 \to \Upsilon_{16}$ to shadow weights w_c
  end if
end for
Clamp $w_c \leftarrow \text{clip}(w_c, -1, +1)$; round all w_c and b to nearest integer
Compute $\Delta(N)$ (Eq. (9)); crystallized iff $\Delta(N) < \varepsilon$
Symbolic extraction (bottom-up):
for each neuron α in topological order **do**
  if α is a merge neuron with $b_\alpha \in \{-1, 0\}$ **then**
    Apply Theorem 4 (exact conjunction / disjunction)
  else if α satisfies Proposition 2 **then**
    Record Łukasiewicz formula for α
  else
    Apply Rule R exhaustively; keep candidate with highest λ
  end if
end for
return Composed formula F over input variables

Training setup. For each dataset and method, we run 10 independent trials with different random seeds and select hyperparameters (architecture, learning rate, and regularization strengths) by grid search on a validation split, reporting test set results. The three methods are: LM with residual connections (LM-Res), STE with Adam, and Proximal with two-phase Adam. All experiments use the smooth crystallization schedule of Section 4.1. Statistical comparisons use the Wilcoxon signed-rank test ($n = 10$, $\alpha = 0.05$).

Evaluation metrics. We report mean accuracy and F_1 score over 10 trials, with standard deviations, together with crystallization rate (fraction of trials achieving $\Delta(N) < 10^{-3}$). Training time (wall clock, seconds) is reported separately.

5.2 Truth table reconstruction

Before turning to real datasets, we verify the methods on synthetic truth tables derived from two target formulas: $f_1 = x_1 \otimes (x_3 \Rightarrow x_6)$ and $f_2 = (x_4 \Rightarrow x_6) \otimes (x_6 \Rightarrow x_2)$.

For f_1, LM achieves exact recovery (MSE $= 0$, $\lambda = 1$) in 1 of 10 trials, reflecting sensitivity to initialization on this formula. STE and Proximal crystallize in all 10 trials but achieve only moderate mean F_1 (0.27 and 0.15 respectively), with Proximal frequently converging to a constant-output solution.

For f_2, LM achieves exact recovery in 5 of 10 trials (mean $F_1 = 0.720$, λ mean $= 0.928$) and converges in 95 iterations on average. Proximal degenerates completely (all 10 trials produce constant-zero output), confirming the failure mode identified in Section 4.4: the ℓ_1 regularizer over-penalizes the discriminative weights before the ternary attraction engages. STE achieves intermediate performance ($F_1 = 0.545$, $\lambda = 0.711$).

These results establish a baseline characterization: LM converges to exact solutions when it crystallizes; STE provides reliable crystallization at moderate formula quality; Proximal is susceptible to degenerate solutions even on simple synthetic problems.

5.3 Crystallization reliability on real datasets

Table 4 reports accuracy and crystallization rates for all six datasets. STE and Proximal achieve 10/10 crystallization on every dataset by construction. LM-Res crystallizes reliably only when a near-integer optimum is reachable within the iteration budget: 10/10 on MONK-3, 9/10 on Heart Disease, but 7/10 on MONK-1/2, 6/10 on Mushroom, and 0/10 on Breast Cancer.

The Heart Disease result (9/10 for LM-Res) is noteworthy because the baseline LM without residual connections achieves 0/10 crystallization on the same problem. This improvement confirms the theoretical prediction: the merge-neuron constraint of Theorem 4 reduces the effective number of weights that must independently crystallize, making the problem more tractable for the second-order solver.

5.4 Classification accuracy

No single method dominates across all datasets. STE leads on Heart Disease (+3.7 percentage points over LM-Res), MONK-1 (+3.5 pp), and MONK-2 (+7.3 pp over LM-Res). LM-Res leads on MONK-3 (+8.2 pp over STE and +24.2 pp over Proximal), with both gaps statistically significant. On Mushroom, all three methods cluster within 0.8 pp (no significant difference).

Table 4: Accuracy (mean ± std, 10 trials) and crystallization rate. * marks significant pairwise Wilcoxon tests ($p < 0.05$). Best accuracy per dataset in **bold**.

	Accuracy (mean ± std)			Crystallization rate		
Dataset	LM-Res	STE	Proximal	LM-Res	STE	Proximal
Mushroom	$\mathbf{.668 \pm .028}$	$.665 \pm .040$	$.660 \pm .000$	6/10	**10/10**	**10/10**
Heart Disease	$.570 \pm .135$	$\mathbf{.607 \pm .101}$	$.600 \pm .000$	9/10	**10/10**	**10/10**
MONK-1	$.565 \pm .218$	$\mathbf{.600 \pm .125}$	$.558 \pm .125$	7/10	**10/10**	**10/10**
MONK-2	$.631 \pm .243$	$\mathbf{.704 \pm .070}$	$.671 \pm .000$	7/10	**10/10**	**10/10**
MONK-3*	$\mathbf{.714 \pm .212}$	$.632 \pm .161$	$.472 \pm .000$	**10/10**	**10/10**	**10/10**
Breast Cancer	$\mathbf{.632 \pm .000}^{\dagger}$	$.572 \pm .120$	$\mathbf{.632 \pm .000}$	0/10	**10/10**	**10/10**

MONK-3: p(LM-Res > Proximal) = 0.016, p(STE > Proximal) = 0.008 (Wilcoxon, $n = 10$).

† LM-Res achieved 0/10 crystallization on Breast Cancer; the reported accuracy is the pre-crystallization continuous-model result. No formula was extracted; this value should not be compared with the crystallized-formula accuracies of STE and Proximal on this dataset.

The degenerate Proximal solutions on five of six datasets — Proximal accuracy equals the majority-class rate with ±0.000 standard deviation, confirming all 10 trials converge to the same fixed point — establish the failure mode as structural rather than a consequence of hyperparameter settings: grid search was conducted over a wide range of λ_s and λ_a values, and the collapse persists. The exception is MONK-1 ($F_1 = 0.147$, non-degenerate), suggesting that the failure is correlated with feature density.

5.5 Formula quality and interpretability

Across all experiments, the only fully representable formula in the sense of Proposition 2 — a formula extractable without any rewriting-algebra approximation — emerges from LM-Res on MONK-3.

MONK-3 exact formula. In trial 1, LM-Res converges in 6 iterations to the fully crystallized network with accuracy 97.2%. The extracted formula is:

$$F = \neg\big(\mathit{body\_shape} \oplus \mathit{jacket\_colour}\big), \tag{15}$$

which can be verified to correctly classify all but 2.8% of the MONK-3 test set (the erroneous labels introduced by the 5% noise). Layer by layer: the inner layer $F^{(1)}$ computes $y = \mathit{body\_shape} \oplus \mathit{jacket\_colour}$ (a disjunction with $n = 0$, $p = 2$, $b = 0$, satisfying Proposition 2 case 2). The shortcut carries $h^{(0)} = y$: in this one-block, one-dimensional network, the shortcut projection also crystallizes to the same linear combination at convergence. The merge neuron therefore receives $F_j^{(1)} = y$ (residual path) and $h_j^{(0)} = y$ (shortcut), and computes $F_j^{(1)} \otimes h_j^{(0)} = y \otimes y$ (conjunction, $b_j = -1$, satisfying Proposition 2 case 1). On binary inputs $y \in \{0, 1\}$, $y \otimes y = \max(0, 2y - 1) = y$, so the conjunction reduces to the identity and the output layer applies the final negation. All neurons satisfy Proposition 2 exactly. STE and Proximal also crystallize on MONK-3 (10/10) but produce denser networks whose inner neurons require the rewriting algebra.

Heart Disease formula extraction. We highlight a Proximal trial that illustrates the framework's capability for clinically interpretable rule extraction. After 321 iterations, one Proximal trial produces a maximally sparse crystallized network on the Heart Disease dataset. Four of the six first-layer neurons carry only zero weights and are discarded by the symbolic extractor. The two active neurons are:

$$n_1 = \psi_{+1}(-\texttt{trestbps},\ -\texttt{oldpeak},\ -\texttt{ca}) = \neg\texttt{trestbps} \otimes \neg\texttt{oldpeak} \otimes \neg\texttt{ca}, \tag{16}$$

$$n_5 = \psi_0(+\texttt{thalach},\ -\texttt{ca},\ -\texttt{cp}{=}4) = \texttt{thalach} \otimes \neg\texttt{ca} \otimes \neg(\texttt{cp}{=}4), \tag{17}$$

where the bias conditions $b = -p + 1 = +1$ in n_1 and $b = n = 0$ in n_5 both satisfy Proposition 2 (conjunctions with three inputs). The output layer combines n_1 and n_5 by: $i_1 = \psi_{+1}(-n_1, -n_5) = \neg n_1 \otimes \neg n_5$. Substituting yields the complete formula:

$$\boxed{\text{DISEASE} = \neg\big(\neg\texttt{trestbps} \otimes \neg\texttt{oldpeak} \otimes \neg\texttt{ca}\big) \otimes \neg\big(\texttt{thalach} \otimes \neg\texttt{ca} \otimes \neg(\texttt{cp}{=}4)\big)} \tag{18}$$

(DISEASE = 1 indicates the presence of coronary artery disease).

Numerical evaluation. Table 5 evaluates Eq. (18) for two representative test patients using the Łukasiewicz conjunction $a \otimes b = \max(0, a + b - 1)$. Patient A has complete vessel occlusion (ca= 1.00), which zeroes both n_1 and n_5, driving the output to 1.00 (disease). Patient B presents with no stenosis, normal blood pressure, and good exercise capacity; the output is 0.00 (healthy). A threshold of 0.5 separates the two correctly. The formula can be verified step by step by any reader who knows the single operator definition $a \otimes b = \max(0, a + b - 1)$.

Table 5: Evaluation trace for Eq. (18) on two test patients (features normalised to $[0, 1]$).

	Patient A (disease)	Patient B (healthy)
ca	1.00	0.00
trestbps	0.64	0.27
oldpeak	0.42	0.00
thalach	0.29	0.79
cp=4 indicator	1	0
$\neg$ca	0.00	1.00
$\neg$trestbps	0.36	0.73
$\neg$oldpeak	0.58	1.00
$\neg$(cp=4)	0.00	1.00
$n_1 = \neg\texttt{t} \otimes \neg\texttt{o} \otimes \neg\texttt{ca}$	$\max(0,\ 0.36{+}0.58{-}1) \otimes 0.00 = 0.00$	$\max(0,\ 0.73{+}1.00{-}1) \otimes 1.00 = 0.73$
$n_5 = \texttt{tal} \otimes \neg\texttt{ca} \otimes \neg(\texttt{cp}{=}4)$	$0.29 \otimes 0.00 \otimes 0.00 = 0.00$	$\max(0,\ 0.79{+}1.00{-}1) \otimes 1.00 = 0.79$
$\neg n_1$	1.00	0.27
$\neg n_5$	1.00	0.21
DISEASE $= \neg n_1 \otimes \neg n_5$	$\max(0,\ 1.00{+}1.00{-}1) = \mathbf{1.00}$	$\max(0,\ 0.27{+}0.21{-}1) = \mathbf{0.00}$

t=trestbps, o=oldpeak, tal=thalach. Values are illustrative from the test set.

Only 5 of the 22 input features received non-zero weights (Table 6). Every selected feature is an established predictor of coronary artery disease in the cardiology literature [1, 21]. Most striking is the selection consistency: `ca` (the number of major vessels with $> 50\%$ stenosis) is the only feature selected in every one of the 10 crystallized Proximal trials, regardless of random initialization. We note that `ca` is universally recognized as the dominant predictor in the Cleveland dataset [21]; the result is therefore not a clinical discovery but a *validation* that the Proximal extractor reliably reproduces established medical knowledge, confirming that the ℓ_1 regularizer does not spuriously suppress clinically established predictors under initialization variance.

Table 6: Features selected in the Heart Disease formula (Eq. 18), with clinical interpretation. "Established": recognized predictor of coronary artery disease [21].

Feature	Clinical meaning	Established
`ca`	Number of major vessels with $> 50\%$ stenosis	✓
`trestbps`	Resting systolic blood pressure (mmHg)	✓
`oldpeak`	ST-segment depression at exercise	✓
`thalach`	Maximum heart rate achieved (bpm)	✓
`cp=4`	Asymptomatic chest pain presentation	✓

Evaluated on the held-out test set (61 patients: 37 healthy, 24 with disease), formula (18) achieves accuracy 85.0%, sensitivity 87.5%, specificity 83.3%, and $F_1 = 0.824$, identifying 21 of 24 disease cases correctly. The formula can be verified analytically by any clinician: it classifies a patient as diseased when neither the triple of low-risk indicators (normal resting blood pressure, no ST depression, no blocked vessels) nor the triple of adequate cardiac reserve indicators (high maximum heart rate, no blocked vessels, symptomatic chest pain) are simultaneously satisfied.

5.6 Training efficiency

Table 7: Mean training time (seconds, 10 trials). Fastest method per dataset in **bold**. p-values from Wilcoxon test ($n = 10$); *: $p < 0.05$.

	Time (s)			p-value		
Dataset	LM-Res	STE	Proximal	LM/STE	LM/PRX	STE/PRX
Mushroom	0.498	1.249	**0.090**	.002*	.002*	.002*
Heart Disease	**0.076**	0.252	0.039	.014*	.064	.002*
MONK-1	**0.212**	0.353	0.522	.131	.084	.160
MONK-2	**0.149**	0.598	0.915	.002*	.002*	.432
MONK-3	**0.033**	0.260	0.338	.002*	.002*	.004*
Breast Cancer	0.450	0.703	**0.148**	.002*	.002*	.002*

LM-Res is the fastest method on four of six datasets, often substantially so: on MONK-3, LM-Res completes in 0.033 s (mean 6.2 iterations) versus 0.260 s for STE (225 iterations) and 0.338 s for Proximal

(48.7 iterations), a 7.9× advantage over STE. The speed advantage reflects the second-order convergence of LM near the solution: once the network is in the basin of a good integer optimum, the Gauss–Newton step reaches it in very few iterations. Proximal is the fastest on Mushroom (5.5× faster than STE) and Breast Cancer (3.0× faster), because aggressive early stopping in Phase 1 terminates quickly — at the cost of solution quality.

6 Related Work

6.1 Rule extraction from neural networks

Extracting symbolic rules from trained neural networks has been studied since the late 1980s. Gallant [3] proposed connectionist expert systems in which threshold units encode conjunctions; the extracted rules are Boolean. KBANN [5] injects domain knowledge into a network and refines it through training, then extracts updated rules via a search over activation patterns. Towell and Shavlik [4] systematized this approach with a weight-based extraction heuristic. All these methods extract rules *post-hoc*: the network is trained as a continuous model and then approximated by a symbolic rule. The approximation introduces an irreducible fidelity gap.

The ŁNN approach is qualitatively different: the symbolic formula is not an approximation to the network but an exact reading of it. When all neurons satisfy Proposition 2, the network *is* the formula. When the rewriting algebra is needed, the approximation is bounded and measurable by the λ-similarity metric rather than being assessed informally.

6.2 Fuzzy and many-valued neural systems

ANFIS [23] and related fuzzy neural architectures learn membership functions and combine them with T-norm rules, producing interpretable models that resemble ŁNNs in spirit. The key difference is that ANFIS membership functions are learned as continuous parameters without a crystallization step; no exact correspondence between neurons and logical connectives is established or enforced. Amato et al. [13] proved that neurons with rational weights can represent rational Łukasiewicz functions, extending the scope of the Castro–Trillas result, but did not provide a training algorithm for integer weights. The present work provides such an algorithm (in three variants) and the corresponding theoretical guarantees.

6.3 Weight quantization

The STE used in Section 4.3 was developed in the context of binarized neural networks [9, 16], where weights are restricted to $\{-1, +1\}$ for hardware efficiency. Ternary weight networks [17] extend this to $\{-1, 0, +1\}$, matching the target domain of crystallization. The objective of quantization in these works is computational efficiency, not logical interpretability: the quantized weights are not interpreted as logical connectives, and no formula extraction procedure is defined. The proximal objective of Section 4.4 is superficially related to ℓ_1-regularized training, but the ternary attraction term $P(w)$ is specific to the discrete target set $\{-1, 0, +1\}$ and is designed to ensure that crystallization produces logically meaningful weights.

6.4 Neuro-symbolic AI

The broader neuro-symbolic AI research programme [24, 25] seeks to combine the learning capabilities of neural networks with the reasoning capabilities of symbolic systems. Recent approaches include neural theorem provers, differentiable logic programs, and logic tensor networks [26]. The common goal is to make neural systems that can reason, or to make symbolic systems that can learn from data.

Łukasiewicz networks and Logical Neural Networks (IBM LNN). Riegel et al. [27] introduced Logical Neural Networks (IBM LNNs), a neural architecture in which every neuron represents a weighted real-valued logical operator (conjunction, disjunction, or negation) using trainable weights in $[0, 1]$. The semantics are akin to Łukasiewicz logic, and the network is trained end-to-end. Three distinctions separate ŁNNs from IBM LNNs. First, IBM LNN weights remain real-valued throughout; no crystallization is defined, and no exact neuron-to-connective correspondence is established. Second, IBM LNN targets knowledge-graph completion and reasoning tasks, not formula extraction from tabular data. Third, ŁNNs provide Proposition 2 as a formal guarantee that a crystallized neuron *is* a Łukasiewicz connective exactly; IBM LNN approximates logical behaviour with continuous weights, without an analogue of Theorem 4.

Logic Tensor Networks and differentiable ILP. Serafini and d'Avila Garcez [28] proposed Logic Tensor Networks (LTNs), which ground first-order logic formulae in neural networks via real-valued satisfaction functions using Łukasiewicz t-norm semantics. Unlike ŁNNs, LTN weights remain continuous and the network does not extract a discrete symbolic formula. Evans and Grefenstette [29] proposed differentiable ILP (∂ILP), which learns first-order logic clauses from noisy data by differentiating through a discrete rule space rather than a neural architecture; ∂ILP does not require integer-weight networks. Manhaeve et al. [30] introduced DeepProbLog, combining neural networks with probabilistic logic programming. DeepProbLog addresses uncertainty via probability distributions over logical facts, whereas Łukasiewicz logic addresses uncertainty via degrees of truth in $[0, 1]$; the two formalisms are complementary.

The ŁNN approach occupies a specific position within this landscape: it is not a hybrid that runs a neural component alongside a symbolic component, but a neural architecture that *is* symbolic after crystallization — the network weights constitute the formula directly. This tight integration is what enables the "interpretability by construction" property of Theorem 4 and distinguishes ŁNNs from all the approaches above.

7 Discussion

7.1 Interpretability by construction versus post-hoc

The distinction between interpretability by construction and post-hoc explanations matters most precisely when it is hardest to verify. A post-hoc explanation is a separate model trained to approximate the decisions of the original model; its fidelity to the original is measurable but never guaranteed to be 1.0 [2]. In the ŁNN framework, when a network is fully crystallized and every neuron satisfies Proposition 2, the formula *is* the model — there is no approximation step, and the verification question reduces to algebraic manipulation of the formula rather than statistical estimation. Theorem 4 extends this guarantee to the residual merge neurons unconditionally.

This property is particularly significant in safety-critical applications. A clinical decision rule that can be written as a Łukasiewicz formula can be audited by domain experts working from the formula alone, without reference to the training data or the training procedure. The step-by-step evaluation in Table 5 illustrates this: a reader who knows only $a \otimes b = \max(0, a + b - 1)$ can verify the classification of any patient independently.

Łukasiewicz formulas versus Boolean decision trees. Both ŁNNs and decision trees are interpretable by construction; the distinction is semantic. A decision tree classifies a continuous feature by thresholding it — $\texttt{trestbps} > 0.6$ is either true or false — where the threshold is determined by an information-gain heuristic with no intrinsic meaning. A Łukasiewicz formula operates on the normalised continuous value directly: $\neg\texttt{trestbps} \otimes \neg\texttt{ca}$ evaluates to $\max(0, (1 - \texttt{trestbps}) + (1 - \texttt{ca}) - 1)$, expressing a graduated joint absence of two risk factors that varies continuously with the input. On normalised features with a physical or clinical scale, this graduated evaluation avoids the arbitrary discretization of decision-tree splits. Furthermore, the McNaughton theorem [11] provides a formal completeness characterisation of the function class expressible by Łukasiewicz formulas (continuous piecewise-linear functions with integer coefficients on $[0, 1]^n$); an equivalent algebraic characterisation for depth-bounded decision trees does not exist. In safety-critical domains where regulations such as IEC 62304 and FDA guidance on AI/ML-based medical devices require traceable, auditable decision rules, a formula with a rigorous logical foundation is preferable to a tree whose structure depends on a training heuristic.

7.2 Limitations

Several limitations of the current framework should be acknowledged. First, the LM solver has quadratic complexity in the number of parameters ($O(p^2)$ for the Jacobian inversion), making it impractical for very large networks; the Mushroom experiment (111 features) already required mini-batch Jacobian approximation [31]. Second, the Proximal failure mode — collapse to constant-output solutions — is structural and is not fully resolved by hyperparameter tuning. Feature pre-selection before Proximal training (using, for example, an initial STE run) is a practical mitigation, but it adds a training step. Third, the expressive power of ŁNNs is bounded by the class of McNaughton functions [10, 11]; it is not known whether every classifiable decision boundary can be approximated to arbitrary accuracy by a ŁNN of bounded depth, and this question is open. Fourth, this paper operates exclusively in the propositional Łukasiewicz framework; the original work [7] treated first-order formulas, and whether the residual block construction extends to the first-order case (where quantifiers introduce cross-input dependencies that the merge neuron structure does not directly accommodate) is an open question. Fifth, all experiments address binary classification; extension to multi-class settings requires a output-layer encoding scheme that is compatible with the symbolic extraction procedure, and this is left for future work.

7.3 Future directions

The theoretical result of Theorem 4 opens several directions. Whether an analogue of the universal approximation theorem holds for many-valued logics — that is, whether finitely many layers of Łukasiewicz connectives can approximate any McNaughton function to arbitrary precision — remains an open question with practical implications for architecture design. The correspondence between ŁNNs and Łukasiewicz logic suggests that other logics based on *residuated lattices* [10] might admit similar neural encodings;

identifying which algebraic structures support interpretable neural implementations is a natural direction. Finally, the interaction between the number of residual blocks and the crystallization rate deserves systematic study: Theorem 4 guarantees that merge neurons crystallize correctly, but its effect on the crystallization dynamics of the inner layers is empirical.

8 Conclusion

The Łukasiewicz neural network framework, established in [7], rests on an exact correspondence between neurons with integer weights and logical connectives. This paper has extended the framework in two directions.

On the theoretical side, we have shown that residual connections can be incorporated into ŁNNs with the symbolic guarantee preserved *by construction*: merge neurons in a Łukasiewicz residual block always satisfy Proposition 2, regardless of the training procedure applied to the inner layers. On the practical side, we have analysed three crystallization strategies, identified and corrected a critical bug in the original LM implementation, and characterized the conditions under which each strategy succeeds or fails.

The experimental results confirm that no single strategy dominates: LM with residual connections recovers exact formulas on structured problems and converges rapidly (under ten iterations on MONK-3); STE provides the most reliable crystallization across all benchmarks; Proximal crystallizes consistently and identifies clinically meaningful features, but collapses on dense datasets. The Heart Disease formula (18), with five variables and direct clinical interpretation, demonstrates that the framework can produce transparent, verifiable diagnostic rules from real medical data.

Together, these results establish Łukasiewicz neural networks as a mature framework for interpretable neuro-symbolic learning, with principled strategies for integer-weight training and a rigorous extension to deeper, residual architectures.

Data Availability Statement

All datasets used in this study are publicly available from the UCI Machine Learning Repository [20] and the Cleveland Heart Disease dataset [21]. Experimental code is available at `https://github.com/carlos-leandro/luknn-replication`.

Ethics Declaration

This study uses publicly available, de-identified datasets. No human subjects research was conducted. No ethical approval was required.

Author Contributions

Carlos Leandro: conceptualization, methodology, software, formal analysis, investigation, writing (original draft, review and editing).

Conflict of Interest Statement

The author declares no conflicts of interest.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

AI Usage Disclosure

Claude (Anthropic) was used for LaTeX formatting assistance and language editing of a draft written by the author. All scientific content, mathematical derivations, and experimental results are the author's own work.

References

[1] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, pp. 206–215, 2019.

[2] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 93:1–93:42, 2018.

[3] S. I. Gallant, "Connectionist expert systems," *Communications of the ACM*, vol. 31, pp. 152–169, 1988.

[4] G. G. Towell and J. W. Shavlik, "Extracting refined rules from knowledge-based neural networks," *Machine Learning*, vol. 13, pp. 71–101, 1993.

[5] ——, "Knowledge-based artificial neural networks," *Artificial Intelligence*, vol. 70, pp. 119–165, 1994.

[6] J. Castro and E. Trillas, "The logic of neural networks," *Mathware and Soft Computing*, vol. 5, pp. 23–27, 1998.

[7] C. Leandro, "Symbolic knowledge extraction using łukasiewicz logics," in *Algorithmic Learning Theory (ALT 2009)*, ser. Lecture Notes in Artificial Intelligence, vol. 5809. Springer, 2009, pp. 477–491, arXiv:1604.03099.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[9] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," in *arXiv preprint arXiv:1308.3432*, 2013.

[10] P. Hájek, *Metamathematics of Fuzzy Logic*. Dordrecht: Kluwer Academic Publishers, 1998.

[11] R. McNaughton, "A theorem about infinite-valued sentential logic," *Journal of Symbolic Logic*, vol. 16, no. 1, pp. 1–13, 1951.

[12] D. Mundici, "Interpretation of AF c^*-algebras in łukasiewicz sentential calculus," *Journal of Functional Analysis*, vol. 65, no. 1, pp. 15–63, 1986.

[13] P. Amato, A. Di Nola, and B. Gerla, "Neural networks and rational łukasiewicz logic," *IEEE Transactions on Fuzzy Systems*, vol. 10, no. 5, pp. 506–510, 2002.

[14] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, 1944.

[15] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963.

[16] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "BinaryNet: Training deep neural networks with weights and activations constrained to $+1$ or -1," in *arXiv preprint arXiv:1602.02830*, 2016.

[17] F. Li, B. Zhang, and B. Liu, "Ternary weight networks," *arXiv preprint arXiv:1605.04711*, 2016.

[18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[19] N. Parikh and S. Boyd, "Proximal algorithms," *Foundations and Trends in Optimization*, vol. 1, no. 3, pp. 127–239, 2014.

[20] D. Dua and C. Graff, "UCI machine learning repository," http://archive.ics.uci.edu/ml, 2019.

[21] R. Detrano, A. Janosi, W. Steinbrunn *et al.*, "International application of a new probability algorithm for the diagnosis of coronary artery disease," *American Journal of Cardiology*, vol. 64, pp. 304–310, 1989.

[22] S. Thrun *et al.*, "The MONK's problems: A performance comparison of different learning algorithms," Carnegie Mellon University, Tech. Rep. CMU-CS-91-197, 1991.

[23] J.-S. R. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, no. 3, pp. 665–685, 1993.

[24] A. d'Avila Garcez, M. Gori, L. C. Lamb, L. Serafini, M. Spranger, and S. N. Tran, "Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning," *Journal of Applied Logics*, vol. 6, no. 4, pp. 611–632, 2019.

[25] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artificial Intelligence Review*, vol. 56, pp. 12 387–12 406, 2023.

[26] P. Hitzler, S. Hölldobler, and A. K. Seda, "Logic programs and connectionist networks," *Journal of Applied Logic*, vol. 2, no. 3, pp. 245–272, 2004.

[27] R. Riegel, A. Gray, F. Rossi, N. Mohamed, J.-A. Feng, J. Palmer, D. Bouneffouf, I. Madan, and S. Trivedi, "Logical neural networks," *arXiv preprint arXiv:2006.13155*, 2020.

[28] L. Serafini and A. d'Avila Garcez, "Logic tensor networks: Deep learning and logical reasoning from data and knowledge," in *NeSy 2016 – 11th International Workshop on Neural-Symbolic Learning and Reasoning*, 2016.

[29] R. Evans and E. Grefenstette, "Learning explanatory rules from noisy data," *Journal of Artificial Intelligence Research*, vol. 61, pp. 1–64, 2018.

[30] R. Manhaeve, S. Dumancic, A. Kimmig, T. Demeester, and L. De Raedt, "DeepProbLog: Neural probabilistic logic programming," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[31] R. Battiti, "First- and second-order methods for learning: Between steepest descent and newton's method," *Neural Computation*, vol. 4, no. 2, pp. 141–166, 1992.